Introduction to Python (Part 1)

COMP 8347

Slides prepared by Dr. Arunita Jaekel

arunita@uwindsor.ca

Python Basics

- Topics
  - Introduction
  - Data Types
  - Arithmetic/Logic Operations

Notable Features

- Many basic data types : numbers (floating point, complex, and unlimited-length long integers), strings, lists, and dictionaries.
- Supports object-oriented programming: with classes and multiple inheritance.
- Supports raising and catching exceptions: cleaner error handling.
- Strongly and dynamically typed data types: mixing incompatible types (e.g. attempting to add a string and a number) causes an exception, errors caught sooner.
- Automatic memory management: frees you from having to manually allocate and free memory in your code.

Notable Feautures

- Elegant syntax: programs easier to read.
- Easy-to-use language: ideal for prototype development.
- Large standard library: supports many common programming tasks e.g. connecting to web servers, regular expressions, file I/O.
- A bundled development environment called IDLE.
- Runs on different computers and operating systems: Windows, MacOS, many brands of Unix, OS/2, ...
- Free software: Free to download or use Python - the language is copyrighted it's available under an open source license.

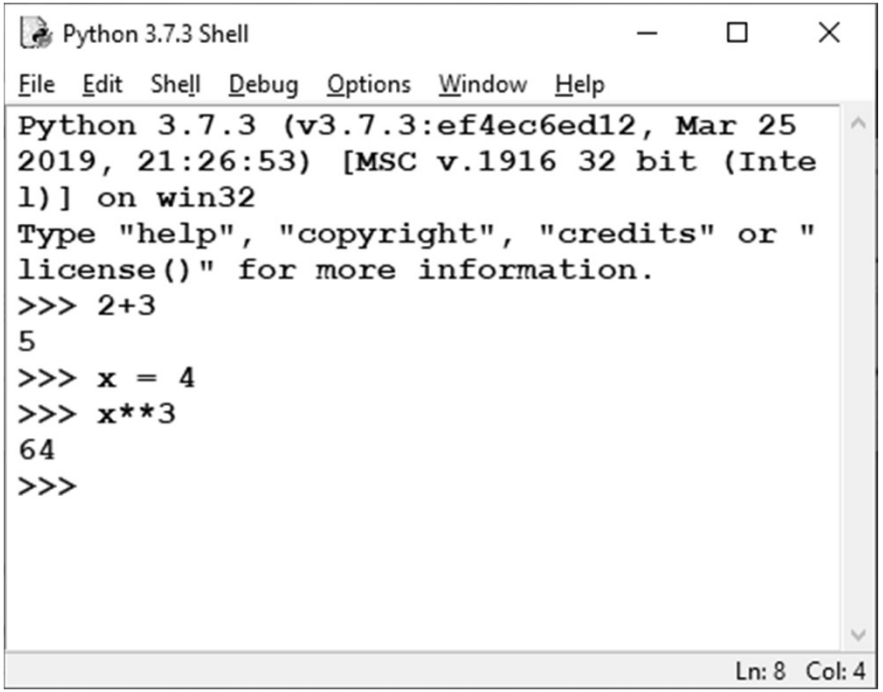
Indentation

- Python does not use brackets to structure code, instead it uses whitespaces
  - Tabs are not permitted.
    - Four spaces are required to create a new block,
  - To end a block simply move the cursor four positions left.
  - An example:

```
def bar(x):
    if x == 0:
        foo()
    else:
        foobar(x)
```

IDLE

- IDLE: Basic IDE that comes with Python
  - Should be available from Start Menu under Python program group.
  - Main "Interpreter" window.
    - Allows us to enter commands directly into Python
    - As soon as we enter in a command Python will execute it display the result.
    - '>>>' signs act as a prompt.



IDLE EXAMPLE

ex1.py - C:/Users/Arunita/OneDr...

File Edit Format Run Options Window Help

```
# My first Python program!!
print('Hello World!')
x = 5
y = 16
z = x + y
print('x + y =', z)
```

Ln: 7 Col: 0

Python 3.7.3 Shell

File Edit Shell Debug Options Window Help

```
>>>
 RESTART: C:/Users/Arunita/OneDrive - Uni
versity of Windsor/8347/slidesF20/ex1.py
Hello World!
x + y = 21
>>>
```

Ln: 12 Col: 4

Numeric Data Types

- *int*: represents positive and negative whole numbers.
  - Written <u>without</u> a decimal point
  - e.g. 5, 258964785663
- *float*: written <u>with</u> a decimal point
  - e.g. 3.0, 5.8421, 0.0, -32.5 etc

Arithmetic Operators

- Basic arithmetic operators: + (addition), - (subtraction), * (multiplication), / (division)
  - / (division) produces floating point value — 15/3 → 5.0
  - // (integer division) truncates any fractional part — 25//3 → 8
  - % (remainder) gives the remainder after integer division. — 25%3 → 1
  - Augmented assignment operators: +=, -=, *=, /=

Basic Input/Output

- Built-in input() function accepts input from user.
  - takes optional string argument to print on console
  - waits for user to type response and hit Enter
  - If no text, user just hits enter: return empty string
  - otherwise, return string containing entered text
  - Example: i = input("Enter an integer: ")
- Built-in print() function for output.
  - Example: print("int = ", i)

ex1.py - C:/Users/Arunita/OneDr...

File Edit Format Run Options Window Help

```
# Basic Input/Output
i = input("Enter an integer: ")
i = int(i)
x = i + 5

print('i =', i)
```

Ln: 6 Col: 14

Python 3.7.3 Shell

File Edit Shell Debug Options Window Help

```
>>>
 RESTART: C:/Users/Arunita/OneDrive - Uni
versity of Windsor/8347/slidesF20/ex1.py
Enter an integer: 7
i = 7
>>> |
```

Ln: 16 Col: 4

Summary

- Python intro
- Numeric data types
- Input/Output

Introduction to Python (Part 2)

COMP 8347

Slides prepared by Dr. Arunita Jaekel

arunita@uwindsor.ca

Python Basics

- Topics
  - Introduction
  - Collection Data Types
    - Strings
    - Lists
    - Dicts
  - Comparison/Logic Operations

Collection Data Types

- Holds a collection of items, which may or may not be of the same type.
- May be *mutable* (e.g. list, set, dict) or *immutable* (e.g. tuple, str)
- *sequence*: a type that supports membership(in), size function (len()), slices ([]) and is iterable. e.g. list, str, tuple
  - https://docs.python.org/3/tutorial/datastructures.html

Mutable vs Immutable

Python 3.8.3 Shell

File Edit Shell Debug Options Window Help

```
>>> list1
[1, 2.5, 'hi', -8]
>>> str1
'Hello World'
>>> list1[0] = 99.9
>>> list1
[99.9, 2.5, 'hi', -8]
>>> str1[0] = 'h'
Traceback (most recent call last):
  File "<pyshell#9>", line 1, in <module>
    str1[0] = 'h'
TypeError: 'str' object does not support item assignment
>>>
```

Ln: 24 Col: 4

Strings

- A collection data type that is ***ordered*** and ***unchangeable*** (***immutable***).
  - e.g. [1], [3, ‘hi’, 4.5, [1,2,3]], []
- The string type in Python is called **str**
- String literals delimited by single or double quotes; e.g. ‘hello’ or “hello”
- Access individual items using the index number and enclosing in square brackets, e.g. str1[0]
- The first element always has index 0

Strings

- Use index([]) and ‘slice’ similar to lists
  - S1 = “A red car ”
  - >>> S1[4]
    - **‘d’**
  - >>> S1[:7]
    - **‘A red c’**
  - >>> S1[2:-6]
    - **‘re’**
  - >>> S1[1] = ‘*’
    - **TypeError: 'str' object does not support item assignment**
  - >>> S1 = ‘something else’
    - # This is ok

S1	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
	A		r	e	d		c	a	r	
	[-10]	[-9]	[-8]	[-7]	[-6]	[-5]	[-4]	[-3]	[-2]	[-1]

String Methods

S1	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
	A		r	e	d		c	a	r	
	[-10]	[-9]	[-8]	[-7]	[-6]	[-5]	[-4]	[-3]	[-2]	[-1]

- S1.count('r')
- **2**
- S1.split()
- **['A', 'red', 'car']**
- S1.replace('r', '*')
- **"A *ed ca* "**
- S1.upper()
- **"A RED CAR "**

NOTE: This is not an exhaustive list

String Methods

S1	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
	A		r	e	d		c	a	r	
	[-10]	[-9]	[-8]	[-7]	[-6]	[-5]	[-4]	[-3]	[-2]	[-1]

- S1.index(‘r’)
- 2
- S1.index(‘x’)
- ValueError
- S1.find(‘r’)
- 2
- S1.find(‘x’)
- -1
- S1.startswith(‘A red’)
- True

NOTE: This is not an exhaustive list

Lists

- A collection data type that is ordered and changeable (mutable).
  - e.g. [1], [3, ‘hi’, 4.5, [1,2,3]], []
- Use [] to index items; similar to strings
- Can have multiple indices, e.g. list1[2][0], list1[-2][-1][-3]

Lists (Examples)

Python 3.8.3 Shell

File Edit Shell Debug Options Window Help

```
>>> list1 = [1, 2.5, 'hello class', [2, 18, 12, 'house'], -8]
>>> list1[1]
2.5
>>> list1[2][0]
'h'
>>> list1[-2][-1][0:3]
'hou'
>>> list1[15]
Traceback (most recent call last):
  File "<pyshell#14>", line 1, in <module>
    list1[15]
IndexError: list index out of range
>>> list1[2:15]
['hello class', [2, 18, 12, 'house'], -8]
>>>
```

Ln: 38 Col: 4

Your Turn …

- L1 = [3, ‘hi’, 4.5, [1,2,3]]
  - >>> L1[1]

    ‘hi’
  - >>> len(L1)

    4
  - >>> L1[3][0]

    1
  - >>> L1[6]

    Error
  - >>> L1[-1]

    [1, 2, 3]

range(n)

- *range(n)*: produces sequence 0, 1, 2, … *n*-1
- range([*i*,]*stop*[, *k*]): sequence starts at *i* (instead of 0) and incremented by *k* (instead of 1)
- range(5) → produces sequence 0, 1, 2, 3, 4
- range(3, 20, 4) → produces 3, 7, 11, 15, 19
- >>> L = [1, 5, 7, 2, 8]
- >>> for i in range(len(L)):

  L[i] = L[i]+10
- >>> L
  - [11, 15, 17, 12, 18]

List Methods

- L = [1,2,3,[4,5,'a','b'], 'hello', '6', 7]
  - L.append([8,9])
  - **[[1, 2, 3, [4, 5, 'a', 'b'], 'hello', '6', 7, [8, 9]]**
  - L += [8,9]
  - **[1, 2, 3, [4, 5, 'a', 'b'], 'hello', '6', 7, 8, 9]**
  - L.index(3)
  - **2**

NOTE: This is not an exhaustive list. We are always starting with initial value of L.

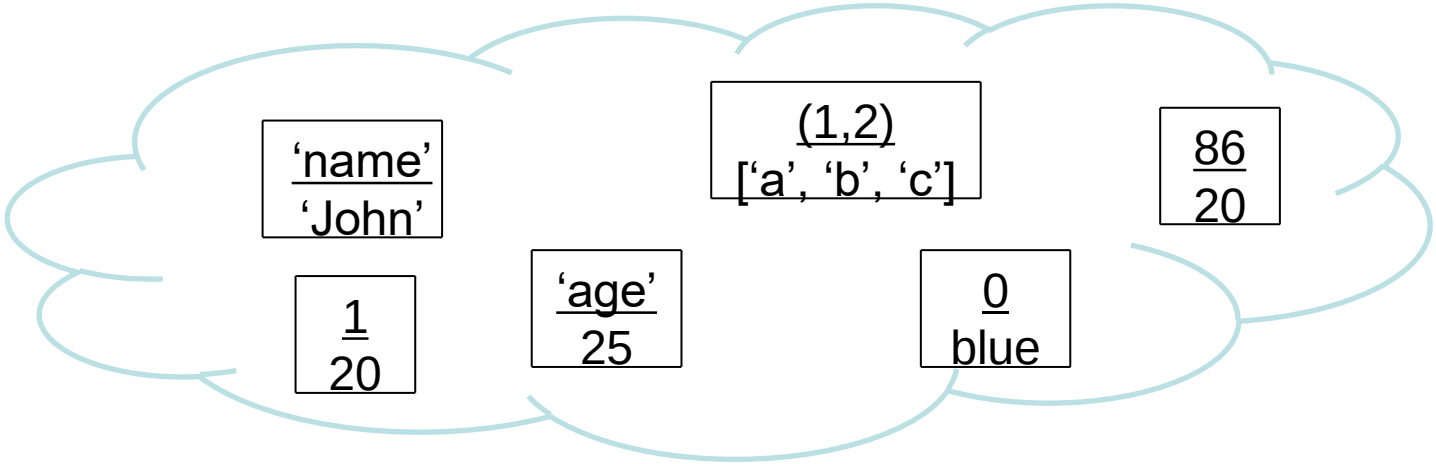
Dictionaries

- *dict*: an ***unordered*** collection of key-value pairs
  - ***mutable***
  - unordered → no notion of index positions
  - keys are unique → a key-value item ***replaces*** existing item with same key
  - duplicate values are fine

- d1 = {“name”: “John”, “age”: 25, (1,2):[‘a’, ‘b’, ‘c’], 0: “blue”, 86: 20, 1:20}

- d2 = dict([(“name”,“John”), (“age”, 25), ((1,2), [‘a’, ‘b’, ‘c’]), (0, “blue”), (86, 20)])

Dictionaries



- **d1 = {"name": "John", "age": 25, (1,2): ['a', 'b', 'c'], 0: "blue", 86: 20}**
- d1["name"]
- **"John"**
- d1[(1,2)]
- **['a', 'b', 'c']**
- d1["blue"]
- **KeyError**
- d1["blue"] = 1
- **# adds new key-value pair "blue":1**

Dictionaries

- **d1 = {'name' : 'John', 'age' : 25, (1,2):['a', 'b', 'c'], 0: 'blue', 86: 20, 1:20, 'blue':1}**
- d1[“blue”]
- 1
- del d1[‘age’]
- # deletes key-value pair “age”: 25
- d1[‘name’] = ‘Ty”
- # replaces key-value pair “name”: “John” with “name”: “Ty”
- d1[d1[“blue”]]
- 20

Membership Operator

- *in*: tests for membership, returns True or False
- *not in*: tests for non-membership, returns True or False
- L = [1, [2,3], “ab”, -23] S = “Python is great!”
  - [2,3] in L
  - **True**
  - 2 in L
  - **False**
  - 2 not in L
  - **True**
  - “on is” in S
  - **True**
  - “eat” not in S
  - **False**

Comparison Operators

- Basic comparison operators: <, <=, ==, !=, >=, >
  - a = 4, b=12
  - a < b → True
  - a == b → False
  - a >= b, a != b, a <= b → (False, True, True)
- Can be chained
  - 2 <= a < b <= 20 → True

Logical Operators

- 3 logical operators: and, or, not
  - and and or use short circuit logic → return operand that determined result, rather than a Boolean value (unless operands are Boolean)
  - Rules for expressions with non-Boolean types
    - int: 0 → False; All other values → True
    - float: 0.0 → False; All other values → True
    - list: [] (i.e. empty list) → False; All other values → True
    - str: ‘’ (i.e. empty string) → False; All other values → True

  - 5 and 2
  - **2**
  - 0 and 2
  - **0**

Your Turn …

- Use short circuit evaluation to determine the results:
  - 5 and 2
  - 2
  - 0 and 2
  - 0
  - 0 or 7 or 8
  - 7
  - 6 or 6 or 9
  - 6
  - not 6
  - False
  - not 0
  - True
  - not ‘0’
  - False

Summary

- Collection data types
  - Range() function
- Boolean operators
  - Membership
  - Comparison
  - Logical

References

- [1] Programming in Python 3 A complete introduction to the python language (2nd Ed) by Mark Summerfield. Addison Wesley 2010.

Python Programming

COMP 8347

Slides prepared by Dr. Arunita Jaekel

arunita@uwindsor.ca

Python Basics

- Topics
  - If statements
  - Loops
  - Exception handling
  - Functions
  - File Input/Output
  - Modules

Control Flow

- Conditional branching
  - IF statement
- Looping
  - While
  - For … in
- Exception handling
- Function or method call

IF Statement

- *suite*: a block of code, i.e. a sequence of one or more statements
- Syntax:

```
if bool_expression1:
   suite1
elif bool_expression2:
      suite2
…
elif bool_expressionN:
      suiteN
else:
      else_suite
```

- No parenthesis or braces
- : used whenever a suite is to follow
- Use <u>*indentation*</u> for block structure

```
 if a < 10:
      print("few")
elif a < 25:
      print("some")
else:
      print("many")
```

While Statement

- Used to execute a suite 0 or more times
  - number of times depends on while loop's Boolean expression.
  - Syntax:

    while bool_expression:

    suite
  - Example:

```
x, sum = 0,0
while x < 10:
        sum += x
        x += 2
print(sum, x)   #What is final value of sum and x
```

Answer: sum=20, x=10

For … in Statement

– Syntax:

for variable in iterable:

suite

– Example: fruits = [‘apple’, ‘pear’, ‘plum’, ‘peach’]

```
for item in fruits:
        print(item)
```

– Alternatively

```
for i in range(len(fruits)):
        print(fruits[i])
```

Break and Continue

ex2.py - C:\Users\Arunita\OneDrive - University of Windsor\8347\slidesF20\ex2.py (3.8.3)

File Edit Format Run Options Window Help

```
# Example using break and continue
# Continue: control returns to top of current loop
# Break: exit from current loop

for num in [11, 8, 3, 25, 9, 16]:
    if num > 20:
        print('exiting loop')
        break   # exit the loop completely
    elif num%2 == 0:
        continue # immediately start next iteration
    print(num)

```

Ln: 7 Col: 28

Iter#	num	num>20?	Num%2==0?	print(num)
0	11	n	n	11
1	8	n	y	
2	3	n	n	3
3	25	y		

Exception Handling

- Functions or methods indicate errors or other important events by ***raising exceptions***.
- Syntax (simplified):

```
try:
    try_suite
except exception1 as variable1:
    exception_suite1
…
except exceptionN as variableN
finally:
    # cleanup
```

- variable part is optional

Exception Handling – Example 1

ex1.py - C:/Users/Arunita/OneDrive - University of Windsor/8347/sli...

File Edit Format Run Options Window Help

```
# Exception Handling
mylist = [7, 8.8, 12, 0, 15]
for i in range(5):
    item = mylist[i**2]
    print('index =', i**2, 'item =', item)
```

Ln: 5 Col: 37

Python 3.7.3 Shell

File Edit Shell Debug Options Window Help

```
 RESTART: C:/Users/Arunita/OneDrive - University of Windsor/83
47/slidesF20/ex1.py
index = 0 item = 7
index = 1 item = 8.8
index = 4 item = 15
Traceback (most recent call last):
  File "C:/Users/Arunita/OneDrive - University of Windsor/8347
/slidesF20/ex1.py", line 4, in <module>
    item = mylist[i**2]
IndexError: list index out of range
>>>
```

Ln: 91 Col: 4

Exception Handling – Example 1

ex1.py - C:/Users/Arunita/OneDrive - University of Windsor/8347/sli...

File Edit Format Run Options Window Help

```
# Exception Handling
mylist = [7, 8.8, 12, 0, 15]
for i in range(5):
    try:
        item = mylist[i**2]
        print('index =', i**2, 'item =', item)
    except IndexError as err1:
        print(err1)
        print(i**2, 'is not a valid index')
```

Python 3.7.3 Shell

File Edit Shell Debug Options Window Help

```
>>>
 RESTART: C:/Users/Arunita/OneDrive - University of Windsor/83
47/slidesF20/ex1.py
index = 0 item = 7
index = 1 item = 8.8
index = 4 item = 15
list index out of range
9 is not a valid index
list index out of range
16 is not a valid index
>>>
```

Ln: 109 Col: 4

Exception Handling – Example 2

Example:

```
s = input('Enter number: ')
try:
    n = float(s)
    print(n, ' is valid. ')
except ValueError as err:
    print(err)
```

- If user enters ‘8.6’ output is: 8.6 is valid
- If user enters ‘abc’, output is:
  ValueError: could not convert string to float: ‘abc'

Functions

- Syntax:

  def functionName(arguments):

  suite

- Parameters are optional. Written as:
  - *positional arguments*: a sequence of comma separated identifiers
  - *keyword argurments*: a sequence of identifier=value pairs
- Every function returns a value
  - either a single value or tuple of values
  - return values can be ignored
  - can leave function at any point using *return* statement
  - no *return* statement or no argument for *return* → returns **None**

Functions

- Function definition:

```
def trianglePerimeter(s1, s2, s3):
    perimeter= s1 + s2 + s3
    return perimeter
```

- **Sample function call: sides = [5, 8, 2]**
  - **result = trianglePerimeter(sides[0], sides[1], sides[2])**
  - **result = trianglePerimeter(*sides) # use sequence unpacking operator (*)**
- **General function for any n-sided polygon:**

```
def perimeter(* sides):
    result = 0
    for s in sides:
        result += s
    return result
```

File Input/Output

- f = open(filename, mode)
  - mode is optional; possible values:
    - ‘w’ = write
    - ‘r’ = read (default)
    - ‘a’ = append
    - ‘rb’(‘wb’) = read (write) in binary
  - Example:
    - f1 = open(“infile.txt”)
    - f2 = open(“outfile.txt”, “w”)
  - f is iterable:

```
for x in f:
    print(x)
```

File Methods

- *f.close()*: closes file object f
- *f.peek(n)*: returns n bytes without moving file pointer position
- *f.read(n)*: read at most n bytes from f
- *f.read()*: read every byte starting from current position
- *f.readline()*: read next line
- *f.readlines()*: read all the lines to the end of file and return them as a list
- *f.write(s)*: write byte/bytearray object s to f in binary mode str object s in text mode.
- *f.writelines(seq)*: write the sequence of objects (strings or byte strings, depending on mode) to f.

Modules

- *Modules*: Contains additional functions and custom data types.
  - Usually a .py file containing Python code; can be written in other languages
  - designed to be imported and used by other programs
- *Packages*: Sets of modules that are grouped together.
- Usage examples:
  - import os
  - import math
  - from os import path

Summary

- Control flow statements
- File I/O
- Modules

17